

AMD GPU Fundamentals & **AI Workloads**

A Comprehensive Academic Reference

Based on AMD Webinar — GPU Fundamentals & Applications in AI Workloads

Prepared as a Personal Study Reference

May 2026

Preface

This document is a structured academic reference compiled from terms and concepts presented during an AMD-organized webinar on GPU Fundamentals and Applications in AI Workloads. The terms have been carefully researched, grouped by thematic relevance, and arranged to form a coherent learning path — from high-level hardware philosophy down to low-level micro-architectural details and computational patterns used in modern AI systems.

The report is divided into five major parts: (I) The AMD Software & Programming Ecosystem, (II) GPU Architecture & Hardware Anatomy, (III) Memory Hierarchy, (IV) Performance Modelling & Profiling, and (V) AI Workloads & Compute Kernels. Each section builds on the previous one, and cross-references are provided wherever concepts interlock.

— Contents —

PART I

The AMD Software & Programming Ecosystem

1. AMD — Advanced Micro Devices

Advanced Micro Devices (AMD) is an American multinational semiconductor company headquartered in Santa Clara, California. Founded in 1969, AMD designs and manufactures microprocessors, GPUs (Graphics Processing Units), FPGAs, and system-on-chip devices used across consumer, professional, high-performance computing (HPC), and AI markets.

In the GPU space, AMD competes directly with NVIDIA and Intel. AMD's GPU strategy is bifurcated into two distinct architectures serving different markets: RDNA (Radeon DNA) for gaming and consumer graphics, and CDNA (Compute DNA) for data-center, HPC, and AI accelerator workloads. This bifurcation reflects AMD's recognition that workload requirements for real-time rasterized graphics and massive-scale floating-point matrix computation are fundamentally different.

AMD's relevance to AI surged with the launch of the Instinct MI-series accelerators (MI100, MI200, MI300) and the accompanying ROCm open-source software platform, positioning AMD as a serious alternative to NVIDIA's CUDA-dominated ecosystem in the AI/ML training and inference space.

Key AMD Product Lines

CPUs: EPYC (server), Ryzen (consumer) | GPUs: Radeon RX (consumer, RDNA), Radeon Pro (workstation), Instinct MI-series (HPC/AI, CDNA) | APUs: Ryzen AI (integrated RDNA + CPU) | MI300X: flagship GPU for large-language-model inference and HPC workloads (2023–2026).

2. CUDA — NVIDIA's Compute Unified Device Architecture

CUDA (Compute Unified Device Architecture) is a parallel computing platform and Application Programming Interface (API) model developed by NVIDIA and released in 2007. Although it is an NVIDIA proprietary technology, understanding CUDA is essential when studying AMD's ecosystem because the entire GPU compute landscape has developed in reaction to — and in competition with — CUDA's dominance.

2.1 Why CUDA Matters

Before CUDA, performing general-purpose computation on a GPU (GPGPU) required developers to express algorithms in terms of graphics primitives (vertex shaders, fragment shaders). CUDA introduced a C/C++ programming model that allowed developers to write GPU code — called kernels — using familiar language syntax, with a small set of extensions for describing parallelism.

CUDA's success stems from:

- A mature, well-documented programming model (threads, blocks, grids, streams)
- A rich ecosystem: cuBLAS (linear algebra), cuDNN (deep neural networks), cuFFT (fast Fourier transforms), TensorRT (inference optimization)

- Deep integration into every major deep-learning framework (PyTorch, TensorFlow, JAX, PaddlePaddle)
- Over 15 years of academic and industrial code written for NVIDIA hardware

CUDA's dominance creates a 'moat': switching from NVIDIA to AMD hardware has historically required porting or rewriting code, which is why AMD invested heavily in HIP and ROCm as portability solutions (see Sections 3 and 4).

2.2 CUDA Programming Model Concepts

CUDA organizes computation into a three-level hierarchy:

Level	CUDA Term	AMD/HIP Equivalent	Description
Thread Grid	Grid	Grid	The entire problem decomposition
Thread Block	Block / CTA	Workgroup / Block	A group of threads sharing local memory
Individual Thread	Thread	Work-item / Thread	Smallest unit of execution
Hardware Execution	Warp (32 threads)	Wavefront (64 threads)	Threads scheduled together in lockstep

Kernels (device functions) are launched from the host (CPU) with a triple-angle-bracket syntax: `kernel<<<gridDim, blockDim>>>(args)`. The GPU scheduler maps thread blocks onto Streaming Multiprocessors (NVIDIA) or Compute Units (AMD). Communication between threads within a block is possible through shared memory (NVIDIA) or LDS — Local Data Share (AMD).

3. HIP — Heterogeneous Interface for Portability

HIP (Heterogeneous Interface for Portability) is AMD's open-source C++ runtime API and kernel language that enables developers to write GPU code that compiles and runs on both AMD GPUs (via ROCm) and NVIDIA GPUs (via CUDA). HIP is the central bridge of AMD's strategy to lower the barrier to GPU portability.

3.1 HIP's Design Philosophy

HIP is deliberately designed to be syntactically close to CUDA. The core programming model is nearly identical: kernels are written with similar constructs, thread indexing uses comparable built-in variables (`hipThreadIdx_x` vs `threadIdx.x`), and memory management follows the same host/device separation. Most CUDA code can be mechanically translated to HIP using AMD's `hipify` tool with minimal manual intervention.

3.2 HIP vs CUDA: Key Differences

Feature	CUDA (NVIDIA)	HIP (AMD)
---------	---------------	-----------

Language	NVIDIA CUDA C++	C++ (HIP dialect)
Compiler	nvcc	hipcc (wraps clang)
Warp Size	32 threads	64 threads (wavefront)
Shared Memory	Shared Memory (__shared__)	LDS (__shared__ in HIP)
Execution Model	SM-based warps	CU-based wavefronts
Atomics	CUDA Atomics	HIP Atomics (near-identical API)
Portability	NVIDIA only	AMD + NVIDIA via hipify

3.3 The hipify Tool

AMD provides hipify-perl and hipify-clang, command-line tools that scan existing CUDA source files and replace NVIDIA-specific API calls and types with their HIP equivalents. For example:

- `cudaMalloc(...)` → `hipMalloc(...)`
- `cudaMemcpy(...)` → `hipMemcpy(...)`
- `__global__` kernels → unchanged (same syntax)
- `threadIdx`, `blockIdx`, `blockDim`, `gridDim` → unchanged

In practice, for straightforward CUDA code, hipify achieves very high conversion rates. However, code that relies heavily on CUDA-specific intrinsics, PTX assembly, or library-specific internals may require manual adaptation.

4. ROCm — Radeon Open Compute Platform

ROCm (Radeon Open Compute platform) is AMD's open-source software stack for GPU compute. It is the AMD equivalent of NVIDIA's CUDA toolkit ecosystem, providing everything required to write, compile, optimize, and run GPU-accelerated applications on AMD hardware.

4.1 ROCm Architecture Stack

ROCm is best understood as a layered stack, from hardware abstractions at the bottom to AI/ML frameworks at the top:

Layer	Component	Role
Hardware	AMD Instinct GPU (MI250, MI300X)	Physical compute silicon
Driver	AMDGPU (kernel driver)	Linux kernel module managing hardware access
Runtime	HIP Runtime / HSA Runtime	Device memory management, kernel dispatch
Compiler	hipcc, LLVM/Clang, AMDGPU backend	Converts HIP/OpenCL to GPU ISA (GCN/CDNA ISA)
Math Libraries	rocBLAS, rocFFT, MIOpen, rocSPARSE	Hardware-optimised linear algebra, DNN primitives

AI Frameworks	PyTorch (ROCm backend), TensorFlow, JAX	High-level ML training and inference APIs
---------------	---	---

4.2 Key ROCm Libraries

ROCm ships with a suite of optimized compute libraries that mirror NVIDIA's CUDA library ecosystem:

- **rocBLAS**: Basic Linear Algebra Subprograms on AMD GPUs. Provides highly tuned implementations of GEMM (General Matrix Multiply), TRSM, and other BLAS Level 1/2/3 routines — the backbone of neural network computations.
- **MIOpen**: AMD's deep learning primitive library (analogous to NVIDIA cuDNN). Provides optimized implementations of convolutions, pooling, batch normalization, RNNs, and activation functions.
- **rocFFT**: Fast Fourier Transform library for AMD GPUs.
- **rocSPARSE**: Sparse linear algebra operations, increasingly important for efficient inference with sparse neural networks.
- **rocRAND**: Random number generation on AMD GPUs.
- **Composable Kernel (CK)**: AMD's high-performance tile-based kernel library for GEMM and convolutions, enabling performance tuning at a lower level than rocBLAS.

ROCm and PyTorch

Since PyTorch 1.8, AMD ROCm is a first-class backend. Users can install PyTorch for ROCm with a single pip command and switch from CUDA to ROCm with minimal code changes. The internal device abstraction layer means that `torch.Tensor` operations look identical — only the device string changes from `'cuda'` to `'rocm'` or `'hip'`.

PART II

GPU Architecture & Hardware Anatomy

5. CPU vs GPU — Architectural Philosophy

The CPU (Central Processing Unit) and GPU (Graphics Processing Unit) represent two fundamentally different approaches to computation, each optimized for a different class of problems. Understanding this distinction is the conceptual foundation upon which all GPU programming models are built.

5.1 CPU Design Philosophy — Latency Optimization

A modern server-class CPU (e.g., AMD EPYC Genoa) contains between 16 and 96 high-performance cores. Each core is engineered for minimal latency on sequential, branch-heavy workloads:

- Deep out-of-order execution pipelines (hundreds of instructions in-flight simultaneously)
- Large per-core caches (typically 32–128 MB of L3 cache shared across cores)
- Branch predictors with extremely high accuracy (>99%)
- Sophisticated prefetchers to hide memory latency
- High clock frequencies (3–5 GHz)

CPUs excel at tasks requiring complex logic, frequent branching, low-latency responses, and sequential dependency chains — operating systems, database engines, web servers, compilers.

5.2 GPU Design Philosophy — Throughput Optimization

A GPU, by contrast, contains thousands of small, simpler processing cores organized into a hierarchy of Compute Units (CUs). Each individual core is less capable than a CPU core but the aggregate throughput is orders of magnitude higher for parallel workloads:

- AMD MI300X: 304 Compute Units, each containing 64 stream processors — 19,456 total shader processors
- Very deep memory bandwidth (up to 5.3 TB/s on MI300X with HBM3 stacked memory)
- Simple in-order execution within each lane (complexity is hidden by wavefront switching)
- Massive register file to support many simultaneous wavefronts (latency hiding)

GPUs excel at data-parallel tasks where the same operation is applied to thousands of independent data elements — matrix multiplication, image processing, Monte Carlo simulations, and neural network computations.

Characteristic	CPU	GPU
Core Count	Tens (16–128)	Thousands (2,000–20,000+)
Clock Speed	3–5 GHz	1–2.5 GHz
Cache Strategy	Large, per-core caches	Small caches, massive bandwidth
Execution Model	Out-of-order, speculative	In-order, many wavefronts
Optimization Target	Latency (single task)	Throughput (many tasks)

Branching Efficiency	Excellent	Poor (divergence costly)
Memory Bandwidth	~300–600 GB/s (DDR5)	~3,200–5,300 GB/s (HBM3)
Peak FP32 TFLOPS	~5–15 TFLOPS	~100–1,300 TFLOPS
Programming Model	Sequential / OpenMP	CUDA / HIP / OpenCL
Best Workloads	OS, DB, compilers, inference	Training, simulation, rendering

The Heterogeneous Computing Model

Modern AI systems use CPUs and GPUs together in a heterogeneous computing model. The CPU acts as the orchestrator (host): it manages data loading, preprocessing, batching, and workflow control. The GPU acts as the accelerator (device): it executes the bulk compute — forward pass, backward pass (gradient computation), optimizer steps. Data is transferred between host DRAM and device HBM via a PCIe bus (or NVLink/Infinity Fabric for tightly coupled systems).

6. GPU Anatomy — Hardware Building Blocks

A GPU is not a monolithic block of transistors but a carefully organized hierarchy of functional units, each designed to work in concert to deliver maximum throughput. This section provides a top-down anatomical tour of an AMD GPU, from the chip die level down to individual execution lanes.

6.1 The Die and Chiplet Architecture

Modern high-end GPUs (especially AMD's CDNA 3 generation, which includes the MI300 series) use a chiplet design: instead of a single monolithic die, the GPU is composed of multiple smaller silicon tiles interconnected via a high-bandwidth die-to-die interface (AMD's Infinity Fabric). This approach improves manufacturing yield and allows mixing GPU compute chiplets (GCDs) with memory chiplets (MCDs) or even CPU chiplets on the same package.

6.2 Shader Engine / Compute Engine

The top-level compute partition in AMD GPUs is the Shader Engine (in RDNA) or the Compute Engine (conceptually, in CDNA). Each Shader Engine contains multiple Compute Units and a geometry pipeline (in RDNA; absent in CDNA which drops rasterization hardware entirely for pure compute).

6.3 Compute Unit (CU) — The Fundamental Building Block

The Compute Unit (CU) is the fundamental building block of AMD GPU compute. All programmable computation happens within CUs. Each CU contains:

- 4 SIMDs (Single Instruction, Multiple Data units), each 16 lanes wide (CDNA) or as configured by the architecture
- A 64 KB Local Data Share (LDS) — on-chip, programmer-managed scratchpad memory
- An L1 instruction cache and L1 scalar data cache
- A Scalar Unit (SU) for handling control flow, address calculations, and uniform operations across a wavefront
- A Vector Unit for the actual floating-point and integer arithmetic on all 64 lanes

- A Branch-and-Message unit for managing wavefront scheduling
- A large Vector Register File (VRF) and Scalar Register File (SRF) to hold the state of multiple in-flight wavefronts

A key point: the CU can hold multiple wavefronts in its register file simultaneously. When one wavefront is stalled waiting for memory, the CU's scheduler instantly switches to another ready wavefront — this is the hardware mechanism for latency hiding (see Section 11).

6.4 Stream Processor / Shader Processor

Within each SIMD unit of a CU, each individual 'lane' is referred to as a stream processor or shader processor. On CDNA2 (MI250X), each CU contains 64 stream processors organized as 4×16 . On CDNA3 (MI300X), the organization is similar but with enhanced matrix capabilities. A stream processor can execute one FP32 operation per clock — but the real power comes from MFMA units for FP16/BF16/FP8 matrix operations.

CU Count in Practice

MI250X (CDNA2): 220 active CUs per GCD, 2 GCDs on one package = 440 CUs total. MI300X (CDNA3): 304 active CUs in a single logical GPU across 8 GCDs. Each additional CU contributes linearly to peak compute throughput for perfectly parallelisable workloads.

7. Wavefronts — AMD's Fundamental Execution Granularity

A wavefront is AMD's term for the smallest unit of parallel execution scheduled by the GPU hardware. It is conceptually equivalent to NVIDIA's warp. Understanding wavefronts is critical for writing efficient GPU kernels because the hardware constraints of wavefront execution directly determine performance.

7.1 What is a Wavefront?

When a kernel (a GPU function) is launched, the problem space is decomposed into work-items (individual threads). The GPU groups 64 consecutive work-items into a wavefront. All 64 work-items in a wavefront execute the same instruction at the same time — this is the SIMT (Single Instruction, Multiple Thread) execution model. The 64 work-items in a wavefront share:

- The same program counter (PC) — they all execute the same instruction
- The same scalar registers (for uniform values like loop indices)
- Individual vector registers (each thread has private state)

Wavefront size in AMD GCN/CDNA architectures: 64 work-items. This contrasts with NVIDIA where warp size is 32. RDNA (gaming) architectures introduced a flexible wave32 mode, but CDNA (compute) stays at wave64.

7.2 Wavefront Divergence

SIMT execution creates a critical performance hazard: branch divergence. When a conditional branch (if/else) causes different threads within a wavefront to take different code paths, the hardware must serialize the execution — first executing the 'true' path for the relevant threads (with the others

masked off), then the 'false' path. This effectively halves or worse the compute efficiency for divergent code.

Best practice: write GPU kernels so that all work-items in a wavefront take the same branch — this is called coherent control flow. Data-dependent branching should be minimized or restructured (e.g., via predication or data sorting).

7.3 Occupancy — Wavefronts Per CU

Occupancy is the ratio of active wavefronts to the maximum number of wavefronts a CU can theoretically hold. Higher occupancy enables more effective latency hiding. The maximum number of wavefronts per CU on CDNA2/3 is typically $10 \text{ wavefronts per SIMD} \times 4 \text{ SIMDs} = 40 \text{ wavefronts per CU}$. However, register pressure limits actual occupancy: if a kernel uses many registers per thread, fewer wavefronts can be resident simultaneously because the register file is shared.

Wavefront Property	CDNA (MI-Series)	RDNA (Radeon RX)	CUDA Warp (NVIDIA)
Size	64 work-items	32 or 64 (Wave32/Wave64)	32 threads
Scheduling	CU scheduler	CU scheduler	SM scheduler
Divergence Cost	High (wave64 serialises more)	Lower in Wave32	Moderate
Max / CU	~40 wavefronts	~20 wavefronts (Wave64)	~32 warps / SM

8. RDNA vs CDNA — AMD's GPU Architecture Split

AMD maintains two distinct GPU microarchitecture families for different markets. Understanding their differences is fundamental to understanding AMD's GPU strategy and the hardware underpinning AI workloads.

8.1 RDNA — Radeon DNA (Consumer/Gaming)

RDNA (introduced in 2019 with the RX 5000 series) is AMD's consumer GPU architecture, designed primarily for:

- Real-time gaming at high frame rates
- Rasterization and ray tracing (DXR/Vulkan RT)
- Display output, video encoding/decoding (VCN engine)
- Compute workloads on consumer hardware (limited by VRAM capacity and bandwidth)

Key RDNA architectural features: Dual-issue execution (Wave32 + Wave64), dedicated ray-accelerators (RDNA 2+), Display Engine, VCN video encoder/decoder, and AMD FidelityFX Super Resolution (FSR) support. RDNA GPUs have VRAM (GDDR6/GDDR6X) rather than HBM — lower bandwidth but lower cost and sufficient for gaming framerates.

8.2 CDNA — Compute DNA (HPC/AI)

CDNA (introduced in 2020 with the MI100) is AMD's data-center GPU architecture, stripped of all gaming-specific hardware and optimized purely for compute throughput:

- No display output, no video encode/decode, no rasterization pipeline
- Much larger HBM (High Bandwidth Memory) with far higher bandwidth than GDDR
- Matrix Fused Multiply-Add (MFMA) units — dedicated hardware for mixed-precision matrix operations
- Infinity Fabric connectivity for multi-GPU communication within a node
- ECC memory for data-center reliability
- Higher thermal design power (TDP) for sustained compute (500–750W per GPU)

CDNA is the architecture powering AMD's Instinct MI-series accelerators used in training and inference of large AI models.

Attribute	RDNA (e.g., RX 7900 XTX)	CDNA (e.g., MI300X)
Target Market	Gaming / Consumer	HPC / AI / Data Center
Memory Type	GDDR6 (up to 24 GB)	HBM3 (up to 192 GB)
Memory Bandwidth	~960 GB/s	~5,300 GB/s
MFMA / Tensor Ops	Limited (AI accelerators)	Dedicated MFMA units
Display Output	Yes (HDMI 2.1, DP 2.1)	No
Ray Tracing	Yes (2nd gen RT cores)	No
Multi-GPU Link	PCIe (limited)	Infinity Fabric (high-BW)
Cooling / TDP	~355W (consumer cooled)	~750W (data-center cooling)
Wave Size	Wave32 default (Wave64 optional)	Wave64 only

9. AMD MI300 — The Flagship AI Accelerator

The AMD Instinct MI300 series, launched in late 2023, represents AMD's most ambitious GPU design and a significant leap in performance for AI inference and HPC workloads. It is built on the CDNA 3 architecture.

9.1 MI300X — Architecture Overview

The MI300X is the GPU-only variant of the MI300 series, targeted at LLM (Large Language Model) inference and large-scale AI training. Its most distinctive feature is its chiplet-based design and unified memory architecture:

- 8 GPU Compute Dies (GCDs) — each a CDNA 3 compute chiplet
- 304 active Compute Units across all 8 GCDs
- 12 Memory Compute Dies (MCDs) — each containing stacked HBM3 memory

- 192 GB total HBM3 memory — the most on any single GPU at launch
- 5.3 TB/s aggregate HBM3 memory bandwidth
- Peak FP8 performance: ~1,307 TFLOPS (with sparsity)
- Peak BF16 performance: ~1,307 TFLOPS (without sparsity)

The 192 GB of unified HBM3 is particularly significant for LLM inference: the entire weights of a 70-billion-parameter model in FP16 require approximately 140 GB of memory, meaning MI300X can hold the full model in GPU memory without offloading — a major bottleneck on GPUs with less memory.

9.2 MI300A — CPU+GPU APU Variant

The MI300A is a unique heterogeneous accelerator that integrates 3 AMD EPYC CPU dies, 6 GPU compute dies, and 8 HBM3 MCDs on a single package with a unified memory address space. CPU and GPU share the same HBM memory pool, eliminating the PCIe bandwidth bottleneck for tightly coupled CPU-GPU algorithms. MI300A is used in the El Capitan supercomputer at Lawrence Livermore National Laboratory.

9.3 CDNA 3 Micro-Architectural Improvements

CDNA 3 (the MI300 generation) introduced several key improvements over CDNA 2 (MI250X):

- FP8 data type support — enables 2× the throughput of FP16 for inference workloads where lower precision is acceptable
- Enhanced MFMA instructions — larger matrix tiles, higher throughput
- Improved Infinity Fabric bandwidth between chiplets
- Second-level cache (L2) significantly expanded

MI300X vs NVIDIA H100

The MI300X competes directly with NVIDIA's H100 80GB SXM5. MI300X advantages: 2.4× more memory (192 vs 80 GB), higher memory bandwidth (5.3 vs 3.35 TB/s). H100 advantages: more mature software ecosystem (CUDA), NVLink 4.0 for multi-GPU all-reduce. For memory-bandwidth-bound LLM inference, MI300X often outperforms H100 per-GPU.

PART III

Memory Hierarchy

10. Memory Hierarchy — From Registers to HBM

Memory hierarchy is one of the most critical aspects of GPU performance. A GPU's processing units can compute far faster than memory can supply data, creating a 'memory wall' that limits practical performance. AMD GPUs address this through a carefully designed multi-level memory hierarchy, each level trading off capacity, latency, bandwidth, and proximity to the compute units.

Understanding the memory hierarchy is essential for writing high-performance GPU kernels: a kernel that fits its working set into LDS or L1 cache runs dramatically faster than one that repeatedly fetches data from HBM.

Memory Level	Location	Capacity	Latency	Bandwidth	Accessible By
Registers	In CU	256 KB / CU (VRF)	~1 cycle	Extremely high	Single thread
LDS (Local Data Share)	In CU	64 KB / CU	~6–7 cycles	~100+ TB/s aggregate	All threads in workgroup
L1 Cache (GL0)	In CU	16–32 KB / CU	~25–30 cycles	~TB/s	Threads sharing CU
L2 Cache (GL1)	Per Shader Engine	~4–16 MB	~100–200 cycles	~TBs of BW	All CUs on die
Infinity Cache (L3/GLC)	On die	Up to 256 MB (RDNA 3)	~150–300 cycles	High hit-BW	All CUs (RDNA only)
HBM (Global Memory)	Off-chip (stacked)	128–192 GB (MI300X)	~600–1000 cycles	~5,300 GB/s	All CUs, host (PCIe)

10.1 Registers — Fastest On-Die Storage

Registers are the fastest storage available to each thread. Every work-item has access to its own private set of registers in the Vector Register File (VRF) of its CU. AMD CDNA GPUs typically provide 512 vector registers (VGPRs) per thread lane, each 32 bits wide. However, the register file is shared across all wavefronts occupying the CU — this is the register pressure trade-off: using more registers per thread reduces the number of wavefronts that can be live simultaneously, reducing occupancy and degrading latency hiding.

Scalar registers (SGPRs) hold values uniform across a wavefront (e.g., base pointers, constants, loop bounds). They are separate from VGPRs and less constrained.

10.2 LDS — Local Data Share

The Local Data Share (LDS) is an on-chip, software-managed scratchpad memory located within each Compute Unit. It is the AMD equivalent of NVIDIA's shared memory. Key characteristics:

- Size: 64 KB per CU (shared among all wavefronts/workgroups currently occupying that CU)
- Latency: approximately 6–7 cycles — dramatically lower than L1 cache (~25 cycles) or HBM (~700+ cycles)
- Bandwidth: the aggregate bandwidth to LDS across all CUs on an MI300X is effectively in the petabytes-per-second range theoretically
- Access: all work-items within the same workgroup (thread block) can read and write LDS
- Persistence: LDS data lives only for the duration of a workgroup's execution

LDS is the primary tool for data reuse within a workgroup. Classic use cases:

- Tiling in matrix multiplication: load a tile of matrix A and matrix B from HBM into LDS, then perform all multiply-accumulates from LDS, then write the result tile back to HBM
- Reduction operations: accumulate partial sums into LDS, then reduce within the workgroup
- Transpose operations: use LDS as a staging buffer to change the memory access pattern from non-coalesced to coalesced

LDS Bank Conflicts

LDS is divided into 32 banks (CDNA). Multiple threads accessing the same bank simultaneously causes a bank conflict, serialising those accesses. Careful data layout (e.g., adding padding columns when loading matrix tiles) avoids bank conflicts. Bank-conflict-free LDS access is a key optimization in tiled GEMM kernels.

10.3 L1 Cache (GL0) — Per-CU Read Cache

The L1 cache (called GL0 in AMD terminology) is a hardware-managed read cache residing within each Compute Unit. Unlike LDS, it is not directly programmable — the hardware automatically caches memory accesses. Key characteristics:

- Size: typically 16 KB per CU (or 32 KB on some configurations)
- Scope: private to each CU — data is not shared with other CUs through L1
- Best for: read-only data with spatial locality (e.g., constant weights accessed by multiple wavefronts on the same CU)
- Write policy: typically write-through or write-combined, not write-back

L1 cache is particularly effective when multiple wavefronts on the same CU access overlapping regions of memory. If data is unique per wavefront or access patterns are irregular, L1 will see poor hit rates and misses will escalate to L2.

10.4 L2 Cache (GL1) — Inter-CU Shared Cache

The L2 cache (GL1) is shared across all CUs in a GPU die. It serves as the last on-die cache before requests must go to HBM. Key characteristics:

- Size: varies by architecture — MI250X: ~8 MB per GCD. MI300X: expanded to ~32 MB per die.

- Latency: ~100–200 cycles
- Bandwidth to CUs: much higher than HBM bandwidth to the chip
- Scope: global to all CUs on a GCD — all CUs can share L2 cached data

L2 cache is the critical buffer between the compute units and HBM. Increasing L2 hit rates (by improving access locality) can dramatically reduce effective memory latency and improve performance for memory-bandwidth-bound kernels.

10.5 Infinity Cache (GL2 / L3 — RDNA Architecture)

Infinity Cache is a large on-die SRAM cache introduced by AMD in RDNA 2 (RX 6000 series) and expanded in RDNA 3. It is specific to AMD's consumer RDNA GPU architecture and functions as a very large L3-equivalent cache:

- RDNA 2 (RX 6000): 128 MB Infinity Cache
- RDNA 3 (RX 7000): up to 96 MB (reduced physical cache but improved efficiency)

Infinity Cache dramatically increases the effective memory bandwidth available to the GPU by capturing a large fraction of memory accesses (particularly the gaming workload's frame buffer and texture data) on-die. For a gaming GPU with a 256-bit GDDR6 bus (which would normally provide ~512 GB/s), Infinity Cache raises the effective bandwidth for cached data to effectively 1–2 TB/s equivalent.

Note: Infinity Cache on CDNA

Infinity Cache is primarily a consumer RDNA feature. AMD's CDNA data-center GPUs (MI250X, MI300X) do not include Infinity Cache — instead they rely on HBM's inherently much higher bandwidth to feed the compute units directly. The MI300X's HBM3 already provides ~5.3 TB/s, making a separate Infinity Cache unnecessary.

10.6 HBM — High Bandwidth Memory

High Bandwidth Memory (HBM) is the primary DRAM solution used in AMD's Instinct data-center GPUs (and AMD's previous generation consumer R9 Fury series). HBM represents a fundamental departure from conventional GDDR memory in that it is stacked vertically beside (or above, in MCM designs) the GPU die and connected via a very wide, short interposer-based interface.

10.6.1 HBM Architecture

An HBM stack consists of multiple DRAM dies stacked on top of each other and connected vertically via Through-Silicon Vias (TSVs). The TSVs allow a very wide memory bus (1024 bits or more per HBM stack vs 32 bits per GDDR6 channel). The stack sits on a silicon interposer (or integrated substrate in chiplet packages like MI300X), which connects it to the GPU die via thousands of micro-bumps.

10.6.2 HBM Generations

Generation	Speed (Gbps/pin)	Bandwidth per Stack	MI Series Used
HBM (Gen 1)	1 Gbps	~128 GB/s per stack	N/A (pre-Instinct)

HBM2	2 Gbps	~256 GB/s per stack	Radeon Instinct MI25
HBM2e	3.2 Gbps	~410 GB/s per stack	MI100, MI210
HBM2e (high)	3.6 Gbps	~460 GB/s per stack	MI250X (6 stacks = 3.2 TB/s)
HBM3	6.4 Gbps	~819 GB/s per stack	MI300X (8 stacks = 5.3 TB/s)
HBM3e	9.6 Gbps	~1.2 TB/s per stack	Future MI-series (in development)

10.6.3 Why HBM Matters for AI

For neural network training and LLM inference, HBM bandwidth is often the primary performance bottleneck (not compute throughput). This is because:

- LLM inference (autoregressive token generation) is bandwidth-bound: at each token step, the entire model's weight matrices must be streamed from HBM to compute units, even though only a small batch (often batch size 1) is processed
- Gradient computation during backpropagation requires reading activations stored during the forward pass from memory
- Mixed-precision training (BF16/FP32) still demands high bandwidth for weight and gradient storage

This is why AMD's investment in HBM bandwidth (5.3 TB/s on MI300X) is a strategic differentiator in the AI accelerator market.

PART IV

Performance Modelling & Profiling

11. The Roofline Model — Performance Analysis Framework

The Roofline Model is a visual and mathematical framework for understanding and predicting the performance of computational kernels on a given hardware platform. Developed by Williams, Waterman, and Patterson (2009) at UC Berkeley, it has become the standard tool for GPU performance analysis and optimization planning.

11.1 Core Concept

The Roofline Model plots attainable performance (in FLOPS — Floating Point Operations Per Second) as a function of arithmetic intensity (in FLOPS per byte of memory traffic). It defines two hard performance ceilings:

- Peak Compute Ceiling: the maximum FLOPS the hardware can deliver, limited by the number of compute units and clock frequency
- Memory Bandwidth Ceiling: the maximum FLOPS achievable given the available memory bandwidth — any kernel that needs to move more bytes than compute operations is limited by bandwidth, not compute

11.2 Arithmetic Intensity

Arithmetic Intensity (AI, not to be confused with Artificial Intelligence) is defined as:

$$\text{Arithmetic Intensity} = \text{FLOPs Performed} / \text{Bytes of DRAM Traffic}$$

Units: FLOPs/Byte. For example, a kernel that performs 100 million floating-point operations while reading/writing 200 MB of data has AI = 0.5 FLOP/Byte.

11.3 Roofline Regions

The Roofline Model divides kernels into two regimes:

- Memory-Bound (left of the ridge point): Arithmetic intensity is low — the kernel spends most of its time waiting for data from memory. Adding more compute units would not help. Optimization must focus on reducing memory traffic (better data reuse, compression, lower precision).
- Compute-Bound (right of the ridge point): Arithmetic intensity is high — compute units are fully utilized and memory bandwidth is not the bottleneck. Optimization must focus on maximizing compute throughput (increasing parallelism, using MFMA units, reducing instruction overhead).

The ridge point is the arithmetic intensity at which peak compute and peak bandwidth limits intersect — this is the minimum AI at which a kernel can be compute-bound.

11.4 MI300X Roofline Example

MI300X key parameters for roofline analysis:

- Peak FP16/BF16 compute (without MFMA): ~383 TFLOPS
- Peak BF16 compute (with MFMA): ~1,307 TFLOPS
- Peak HBM3 bandwidth: ~5,300 GB/s = 5.3 TB/s

Ridge point (BF16 MFMA) = 1,307 TFLOPS / 5.3 TB/s $\approx$ 247 FLOP/Byte. This means a BF16 GEMM kernel needs an arithmetic intensity of at least 247 FLOP/Byte to be compute-bound. In practice, a matrix multiplication with large matrices ($N \gg 1024$) achieves this; small batch inference does not.

Using the Roofline in Practice

Step 1: Profile your kernel to measure actual FLOPS and actual bytes read/written (use AMD rocprof or Omnipert). Step 2: Plot the result on the roofline. Step 3: If memory-bound, optimize for data reuse (tiling, vectorisation, prefetch). If compute-bound, look for idle lanes (poor occupancy, divergence, or un-utilized MFMA units).

12. Latency Hiding — Tolerating Memory Latency Through Multi-Threading

Latency hiding is one of the most fundamental performance techniques in GPU computing. It is the mechanism by which a GPU tolerates the long latency of HBM memory accesses (600–1,000 cycles) by keeping the compute units busy with other work while waiting for data to arrive.

12.1 The Latency Problem

If a single-threaded CPU program issues a memory load, execution stalls until the data arrives — but a large cache and out-of-order execution hide most of this. A GPU CU has much smaller caches and simple in-order execution within each wavefront. A cache miss to HBM costs ~700 clock cycles. Without latency hiding, those 700 cycles would be completely wasted.

12.2 The Hardware Solution: Multiple Wavefronts

AMD GPU CUs solve this by holding many wavefronts in their register file simultaneously. When Wavefront A issues a memory load and must stall waiting for the result:

1. The CU scheduler immediately switches to Wavefront B (which is ready to execute)
2. Wavefront B runs until it stalls (or completes)
3. The scheduler moves to Wavefront C, D, E...
4. By the time all other wavefronts have stalled or run, Wavefront A's data has likely arrived
5. Wavefront A resumes execution

This is zero-overhead context switching — because each wavefront has its own dedicated registers, there is no state to save or restore. Switching takes exactly one cycle.

12.3 Occupancy and Latency Hiding

The effectiveness of latency hiding is directly proportional to occupancy (the number of active wavefronts per CU). With higher occupancy:

- More wavefronts are available to fill the time while waiting for memory
- Compute utilization stays higher
- Effective memory latency (as seen by throughput) is reduced

If occupancy is low (e.g., only 2 wavefronts active per CU due to high register usage), the CU may run out of ready wavefronts and stall, leaving execution units idle — this is called a pipeline stall.

The relationship is: Latency Tolerance = Number of Active Wavefronts × Instructions per wavefront before a memory instruction. A kernel should have sufficient computation between memory accesses to give memory requests time to complete before the result is needed.

Key Insight

Latency hiding explains why GPU programs should prefer high occupancy even when individual wavefront performance is slightly reduced. 10 wavefronts running at 80% individual efficiency will almost always outperform 2 wavefronts running at 100% individual efficiency, because the 10-wavefront case hides memory latency far more effectively.

13. Profiling AMD GPU Workloads

Profiling is the process of measuring a GPU program's runtime behavior — FLOP throughput, memory bandwidth utilization, wavefront occupancy, cache hit rates, and more — to identify performance bottlenecks. AMD provides a robust suite of profiling tools in the ROCm stack.

13.1 rocprof — Command-Line Profiler

rocprof is AMD's primary hardware performance counter profiling tool. It intercepts HIP/ROCm kernel launches and collects hardware counter data including:

- SQ_INSTS_VALU: Vector ALU instruction count
- TA_BUSY: Texture Addresser busy cycles (memory access activity)
- L2CacheHit: L2 cache hit rate
- FETCH_SIZE: Total bytes fetched from HBM
- TCC_HIT: L2 (TCC) cache hit count

rocprof can also produce timeline traces viewable in Chrome's trace viewer or Perfetto UI.

13.2 Omniperf — Detailed Microarchitectural Profiler

Omniperf (formerly rocProf-compute) is AMD's advanced profiling tool that provides a Roofline analysis and detailed microarchitectural breakdowns of GPU kernel performance. It produces a comprehensive report including:

- Speed-of-Light analysis: shows each kernel as a percentage of peak compute, peak bandwidth, and peak L2 bandwidth
- Wavefront occupancy statistics
- LDS, L1, L2 hit rates and access patterns
- MFMA utilization vs vector ALU utilization
- Instruction mix breakdown

13.3 ROCm Systems Management Interface (rocm-smi)

rocm-smi is a system-level monitoring tool providing real-time GPU utilization, memory usage, temperature, power consumption, and clock frequencies — similar to NVIDIA's nvidia-smi.

13.4 Profiling Workflow for AI Kernels

6. Identify the slow kernel: use high-level framework profiling (PyTorch Profiler) to find which GPU kernels consume the most time
7. Run rocprof or Omnipperf on the bottleneck kernel
8. Check the Roofline: is the kernel memory-bound or compute-bound?
9. If memory-bound: increase data reuse (tiling), use lower precision (FP16/BF16/FP8), coalesce memory accesses
10. If compute-bound: increase MFMA utilization, reduce instruction overhead, optimize occupancy
11. Re-profile after changes to verify improvement

PART V

AI Workloads & Compute Kernels

14. Neural Networks — The Driving Force of GPU Computing

Artificial Neural Networks (ANNs) are computational models loosely inspired by biological neural systems in the brain. They consist of interconnected layers of mathematical functions (neurons) that learn to transform input data into desired outputs by adjusting their internal parameters (weights) through a training process.

14.1 Why Neural Networks Are Ideal for GPUs

Neural network computation is almost entirely composed of two operations repeated billions of times: matrix multiplication (fully-connected layers, attention in transformers) and convolution (vision models, CNNs). Both operations are:

- Massively data-parallel: each output element is independent of others
- Regular in memory access (predictable patterns, good for cache prefetching)
- Compute-intensive at large batch sizes (high arithmetic intensity)

These properties map almost perfectly to GPU architecture. A GPU's thousands of compute lanes can process thousands of neurons in parallel, and the MFMA hardware can accelerate the matrix math at extraordinary speed.

14.2 Training vs Inference

Aspect	Training	Inference
Goal	Learn model weights from data	Apply learned weights to new data
Computation	Forward + Backward pass + Optimizer	Forward pass only
Memory	Activations, gradients, weights, optimizer states	Weights (+ activations for seq)
Batch Size	Large (32–8192)	Small (1–32, latency-critical)
Bottleneck	Compute (MFMA, TFLOPS)	Memory bandwidth (weight loading)
Precision	BF16/FP16 for compute, FP32 for master weights	INT8/FP8/BF16 for inference
Hardware	Multi-GPU clusters (MI300X × 8+)	Single GPU or CPU-offload

14.3 Common Neural Network Architectures

- Fully Connected Networks (MLPs): every neuron connects to every neuron in the next layer — pure matrix multiplication
- Convolutional Neural Networks (CNNs): use sliding window convolution filters — suited for vision tasks (image classification, object detection, segmentation)
- Recurrent Neural Networks (RNNs, LSTMs, GRUs): sequential models for time-series and text — largely superseded by Transformers
- Transformer Networks: based on self-attention mechanisms — dominate NLP (GPT, BERT, LLaMA, Mistral) and increasingly vision (ViT, CLIP, DinoV2)
- Diffusion Models: iterative denoising networks for generative tasks (Stable Diffusion, DALL-E) — compute-intensive during inference

15. Matrix Multiplication (MatMul / GEMM) — The Core Primitive

Matrix multiplication — often called GEMM (General Matrix Multiply) in library terminology — is the single most important computational primitive in all of deep learning. Understanding GEMM performance is understanding neural network performance on GPUs.

15.1 Mathematical Definition

Given matrices A of shape $[M \times K]$ and B of shape $[K \times N]$, the output $C = A \times B$ has shape $[M \times N]$, where each element $C[i,j] = \text{sum over } k \text{ of } A[i,k] * B[k,j]$. The total number of multiply-add operations is $M \times N \times K$ multiplications and $M \times N \times K$ additions = $2 \cdot M \cdot N \cdot K$ FLOPs.

15.2 Arithmetic Intensity of GEMM

The arithmetic intensity of GEMM scales with matrix size. For square matrices of size N:

- FLOPs: $2 \cdot N^3$
- Bytes (naively, if loading A, B once and writing C once): $3 \times N^2 \times \text{element_size}$
- $AI = 2 \cdot N^3 / (3 \cdot N^2 \cdot 4) = N/6$ for FP32

For large N (e.g., $N = 4096$): $AI \approx 4096 / 6 \approx 682$ FLOP/Byte — well above the MI300X ridge point of ~ 247 FLOP/Byte. Large GEMMs are compute-bound on modern hardware. Small GEMMs (small M, as in batch-size=1 inference) are memory-bound because weights must be streamed but few output elements are computed.

15.3 Tiled GEMM and the Role of LDS

Naively, each output element $C[i,j]$ would require loading a full row of A and column of B from HBM. The tiled GEMM algorithm dramatically reduces HBM traffic by loading tiles into LDS and reusing them for all output elements within the tile:

12. Partition A, B into tiles of size $[BM \times BK]$ and $[BK \times BN]$ respectively
13. For each tile step k: load tile of A and tile of B from HBM into LDS
14. Perform all $BM \times BN \times BK$ multiply-accumulates using data in LDS
15. Accumulate result into a register-resident C tile
16. After all k steps, write the C tile result to HBM

This reduces HBM reads from $O(M \cdot N \cdot K)$ to $O(M \cdot N \cdot K / \text{tile_size})$ — a factor of tile_size improvement in memory bandwidth requirements.

15.4 GEMM and Neural Network Layers

Layer Type	GEMM Shape	M	K	N
Linear / Dense	$\text{Output} = \text{Input} \times \text{Weight}^T$	$\text{Batch} \times \text{Seq_Len}$	In_features	Out_features
Attention Q/K/V proj	XW^T for Q,K,V matrices	$\text{Batch} \times \text{Seq_Len}$	Hidden_dim	$\text{Head_dim} \times \text{Heads}$
Attention Score (QK^T)	Per-head score matrix	$\text{Batch} \times \text{Heads} \times \text{Seq}$	Head_dim	Seq_Len
MLP (FFN) layer	Two sequential GEMMs	$\text{Batch} \times \text{Seq}$	Hidden	$4 \times \text{Hidden}$

16. Convolution — The Core Primitive of Computer Vision

Convolution is the defining operation of Convolutional Neural Networks (CNNs) and remains critical in many vision and signal-processing applications. While transformers have partially displaced CNNs in some vision tasks, convolutional layers still appear in diffusion model U-Nets, image segmentation models, and efficient mobile architectures.

16.1 2D Convolution Operation

A 2D convolution applies a small filter (kernel) of size $[\text{Kh} \times \text{Kw}]$ to every spatial position of an input feature map of shape $[\text{N} \times \text{Cin} \times \text{H} \times \text{W}]$, producing an output of shape $[\text{N} \times \text{Cout} \times \text{Hout} \times \text{Wout}]$. Each output channel uses a different filter. The filter weights are learned during training.

FLOPs for a convolutional layer: $2 \times \text{N} \times \text{Cout} \times \text{Hout} \times \text{Wout} \times \text{Cin} \times \text{Kh} \times \text{Kw}$. For a typical ResNet-50 conv layer (3 × 3 kernel, 256 channels, 56 × 56 spatial): ~170 million FLOPs per image.

16.2 Convolution Implementations on GPU

Direct convolution is rarely implemented as written mathematically. GPU-optimized libraries (MIOpen, cuDNN) use several algorithms:

- implicit GEMM: the convolution is reformulated as a matrix multiplication using the im2col transform (reshaping the input patches into columns), enabling use of highly optimized GEMM kernels. This is the most common approach for standard kernel sizes.
- Winograd algorithm: a mathematically equivalent transform that reduces the number of multiplications (at the cost of more additions) for small kernels (3×3, 5×5) — often 2–4× fewer FLOPs than direct convolution
- FFT-based convolution: for very large kernels, convolution in the frequency domain via Fast Fourier Transform is more efficient, since convolution becomes element-wise multiplication in frequency space

16.3 Depthwise and Grouped Convolutions

Standard convolution is compute-intensive but parameter-efficient. Depthwise convolution applies one filter per input channel (no mixing between channels), dramatically reducing FLOPs and parameters. It is used in MobileNet and EfficientNet architectures for mobile-friendly inference.

Grouped convolution partitions channels into G groups, applying separate convolutions per group — used in ResNeXt and ShuffleNet architectures. Both depthwise and grouped convolutions have lower arithmetic intensity than standard convolutions and can be memory-bandwidth-bound.

17. MFMA — Matrix Fused Multiply-Add

MFMA (Matrix Fused Multiply-Add) is AMD's hardware instruction for accelerating matrix multiplication using dedicated on-chip matrix engines within each Compute Unit. It is AMD's equivalent of NVIDIA's Tensor Cores, and it is the reason modern AMD Instinct GPUs can achieve dramatically higher throughput for neural network workloads than their raw SIMD FP32 throughput would suggest.

17.1 What MFMA Does

A single MFMA instruction performs a small matrix multiply-accumulate on a fixed-size matrix tile. For example, on CDNA2 (MI250X), a `v_mfma_f32_16x16x16_f16` instruction takes:

- Two 16×16 FP16 input matrices (A and B tiles, stored in VGPR registers across the 64 lanes of a wavefront)
- One 16×16 FP32 accumulator matrix
- Performs: Accumulator $+= A \times B$ in FP16 with FP32 accumulation
- All $16 \times 16 \times 16 = 4096$ multiply-accumulates in a single instruction

The key insight: instead of 4,096 individual FMA (Fused Multiply-Add) instructions (one per lane over multiple wavefront cycles), MFMA achieves the same computation in a single instruction using dedicated matrix datapath hardware — achieving a multiplier of 4–8× on effective throughput versus element-wise SIMD operations.

17.2 MFMA Tile Sizes (CDNA2 / CDNA3)

Instruction	A/B Type	Tile Size (M×N×K)	FLOPs per Instr.	Accumulator Type
<code>v_mfma_f32_32x32x8_f16</code>	FP16	$32 \times 32 \times 8$	16,384	FP32
<code>v_mfma_f32_16x16x16_f16</code>	FP16	$16 \times 16 \times 16$	8,192	FP32
<code>v_mfma_f32_32x32x8_bf16</code>	BF16	$32 \times 32 \times 8$	16,384	FP32
<code>v_mfma_f32_16x16x16_bf16</code>	BF16	$16 \times 16 \times 16$	8,192	FP32
<code>v_mfma_f64_16x16x4_f64</code>	FP64	$16 \times 16 \times 4$	2,048	FP64
<code>v_mfma_f32_32x32x16_fp8</code> (CDNA3)	FP8	$32 \times 32 \times 16$	32,768	FP32

17.3 MFMA and the Kernel Design Challenge

While MFMA offers enormous throughput, actually achieving near-peak MFMA utilization in a kernel is non-trivial. Efficient MFMA kernels must:

- Keep MFMA units continuously fed by staging data in LDS ahead of time (double-buffering: load next tile from HBM into LDS while computing on current tile with MFMA)
- Use FP16 or BF16 (or FP8 on CDNA3) input types to maximize MFMA throughput
- Carefully manage register pressure so occupancy remains sufficient for latency hiding
- Avoid scalar operations (control flow, address math) that cannot be pipelined with MFMA

The AMD Composable Kernel (CK) library and the Triton GPU compiler (which has an AMD ROCm backend) are the primary tools for writing MFMA-optimized kernels without hand-writing assembly.

18. Kernel Design — Writing Efficient GPU Programs

A GPU kernel is a function executed in parallel by thousands of GPU threads. The design of a kernel — how it decomposes work, manages memory, and exploits hardware features — determines whether it runs at 10% or 90% of peak hardware performance. Kernel design is both a science (roofline analysis, occupancy theory) and an engineering art (micro-optimization, profiler feedback loops).

18.1 Work Decomposition

The first step in kernel design is deciding how to map the problem to a grid of workgroups (thread blocks) and work-items (threads). For a matrix multiplication $C = A \times B$:

- Each workgroup computes one output tile of C (e.g., 128×128 elements)
- Within the workgroup, each wavefront is responsible for a sub-tile
- The grid is sized so the total number of output tiles covers the entire matrix C

18.2 Memory Access Coalescing

GPU memory controllers serve global memory in transactions aligned to cache-line boundaries (typically 128 bytes). If threads in a wavefront access memory addresses that fall within a single 128-byte cache line, the hardware issues one memory transaction — this is coalesced access. If each thread accesses a random address, 64 separate transactions are needed — 64× worse bandwidth utilization.

Rule: threads with consecutive IDs should access consecutive memory addresses. For row-major matrices, iterating across the column dimension (innermost) for consecutive thread IDs achieves coalescing.

18.3 Double Buffering — Overlapping Compute and Memory

Double buffering is a critical technique for maximising MFMA utilization. Instead of the naive pattern of 'load tile → compute → load tile → compute', double buffering uses two LDS buffers:

17. Buffer A is being computed on (MFMA running on data in Buffer A)
18. Simultaneously, Buffer B is being loaded from HBM (async prefetch)
19. When compute finishes on A, swap buffers: A is now the load buffer, B is the compute buffer

This hides HBM load latency behind MFMA computation, enabling the hardware to approach its peak MFMA throughput.

18.4 Precision Selection and Mixed-Precision

Precision choice dramatically affects both performance and accuracy:

Precision	Bits	Range	AI Use Case	MI300X Throughput
FP64	64-bit	Full scientific precision	Scientific HPC, optimizer	~163 TFLOPS
FP32	32-bit	Full neural net precision	Master weights, accumulators	~163 TFLOPS
BF16	16-bit (brain float)	Same exponent range as FP32	Training forward/backward	~1,307 TFLOPS (MFMA)
FP16	16-bit (IEEE)	Smaller range, higher precision	Training (with loss scaling)	~1,307 TFLOPS (MFMA)
FP8	8-bit	Very limited range	Inference, fine-tuning (CDNA3)	~2,614 TFLOPS (MFMA)
INT8	8-bit integer	Integer only	Post-training quantization	High throughput

18.5 Triton — Productivity Language for GPU Kernels

Triton is an open-source language (developed by OpenAI, with AMD ROCm support) that enables writing high-performance GPU kernels in Python-like syntax, without hand-writing CUDA/HIP or assembly. Triton automatically handles:

- Tile-based work decomposition
- LDS usage for tiled GEMM and convolution
- Coalesced memory access generation
- MFMA instruction generation (via backend lowering to HIP/CUDA)

Triton is used extensively in PyTorch 2.0's torch.compile path (via Inductor backend) and has emerged as the de-facto productivity layer for writing custom AI kernels on both NVIDIA and AMD hardware.

19. Synthesis — How All Concepts Connect

The terms covered in this report do not exist in isolation — they form an interconnected ecosystem. This section provides a high-level map of how the concepts relate to each other.

The Full Stack: From Algorithm to Silicon

Neural Network Training / Inference (LLMs, CNNs) → Implemented using PyTorch or JAX (AI frameworks) → Which call MIOpen / rocBLAS kernels (or custom Triton kernels) → Which execute MFMA instructions on CDNA 3 Compute Units → Where Wavefronts hide memory latency by context-switching → Accessing data from LDS → L1 → L2 → HBM3 → Performance analysed using the Roofline Model → Profiled via Omnipperf → Optimised through Kernel Design (tiling, double-buffering, precision) → All made portable via HIP → Running on ROCm → On AMD MI300X silicon.

The following table summarises the five-part structure and key interconnections:

Concept	Category	Depends On / Related To
AMD	Company/Platform	ROCm, CDNA, RDNA, MI300
CUDA	Programming Model	NVIDIA hardware; reference for HIP
HIP	Programming Model	ROCm, CUDA (portability layer)
ROCm	Software Stack	HIP, rocBLAS, MIOpen, AMD drivers
CPU vs GPU	Architecture Philosophy	GPU Anatomy, Latency Hiding, Wavefronts
GPU Anatomy	Hardware	CU, SIMD, LDS, Register File, Wavefronts
Compute Unit	Hardware Building Block	Wavefronts, LDS, L1, MFMA units
Wavefronts	Execution Model	CU, Latency Hiding, Kernel Design, Divergence
RDNA vs CDNA	Architecture Split	AMD, MI300, Infinity Cache, MFMA
MI300	Specific Hardware	CDNA, HBM3, MFMA, CDNA3, Kernel Design
LDS	Memory Level	CU, Kernel Design, Tiling, GEMM, Latency Hiding
L1 / L2 Cache	Memory Level	CU, Memory Hierarchy, Roofline, Profiling
Infinity Cache	Memory Level (RDNA)	RDNA, GDDR6, Memory Hierarchy
HBM	Memory Technology	MI300, Bandwidth, Roofline, LLM Inference
Roofline Model	Performance Analysis	Arithmetic Intensity, HBM BW, MFMA, Profiling
Latency Hiding	Performance Technique	Wavefronts, Occupancy, Registers, LDS
Profiling	Optimization Tool	Roofline, rocprof, Omnipperf, Kernel Design

MatMul / GEMM	Compute Primitive	MFMA, LDS, Tiling, Neural Networks, rocBLAS
Convolution	Compute Primitive	GEMM (im2col), MIOpen, CNNs, Kernel Design
Neural Networks	AI Workload	MatMul, Convolution, MFMA, MI300, Training/Inference
MFMA	Hardware Instruction	CDNA, CU, Kernel Design, MatMul, Precision
Kernel Design	Software Optimization	All hardware concepts, Roofline, LDS, MFMA, Wavefronts

20. Glossary of Key Terms

Term	Full Name / Meaning
AI (context-dependent)	Arithmetic Intensity (FLOP/Byte) — in performance analysis; Artificial Intelligence — in general
ALU	Arithmetic Logic Unit — performs math and logic operations
APU	Accelerated Processing Unit — AMD integrated CPU+GPU chip
BF16	Brain Float 16 — 16-bit float with same exponent range as FP32, better for training
BLAS	Basic Linear Algebra Subprograms — standardized linear algebra routines
CK	Composable Kernel — AMD high-performance tile-based kernel library
CTA	Cooperative Thread Array — CUDA term for thread block / AMD workgroup
CDNA	Compute DNA — AMD GPU architecture for HPC/AI data-center workloads
CU	Compute Unit — fundamental building block of AMD GPU compute
CUDA	Compute Unified Device Architecture — NVIDIA parallel computing platform
DXR	DirectX Raytracing — Microsoft API for real-time ray tracing
ECC	Error-Correcting Code — memory protection in data-center GPUs
FMA	Fused Multiply-Add — compute $a*b+c$ in one instruction, avoiding rounding errors
FP8/16/32/64	8/16/32/64-bit floating-point number representation
GCD	GPU Compute Die — individual GPU chiplet in AMD MCM designs
GDDR6	Graphics DDR6 — high-speed DRAM used in consumer GPUs (vs HBM in compute GPUs)
GEMM	General Matrix Multiply — the core linear algebra routine for deep learning
HBM	High Bandwidth Memory — stacked DRAM with very wide interface, used in AI accelerators
HIP	Heterogeneous Interface for Portability — AMD's GPU programming language/API
HSA	Heterogeneous System Architecture — AMD's unified CPU+GPU computing framework
ISA	Instruction Set Architecture — the hardware instruction language of a processor
LDS	Local Data Share — on-chip scratchpad memory per AMD Compute Unit
LLM	Large Language Model — transformer-based AI models (GPT, LLaMA, Gemini, Claude)
MCM	Multi-Chip Module — packaging multiple dies on one substrate (MI300 approach)
MCD	Memory Compute Die — HBM memory chiplet in AMD MI300 package
MFMA	Matrix Fused Multiply-Add — AMD instruction for hardware-accelerated matrix operations
MIOpen	AMD deep learning primitive library (analogous to NVIDIA cuDNN)

occupancy	Ratio of active wavefronts to maximum possible wavefronts per CU
PCIe	Peripheral Component Interconnect Express — interface connecting CPU and GPU
RDNA	Radeon DNA — AMD GPU architecture for gaming and consumer graphics
rocBLAS	ROCm BLAS library — highly optimized linear algebra for AMD GPUs
rocprof	ROCm profiler — hardware counter profiling tool for AMD GPUs
ROCm	Radeon Open Compute — AMD's open-source GPU compute software platform
SIMD	Single Instruction Multiple Data — one operation applied to many data elements in parallel
SIMT	Single Instruction Multiple Threads — GPU extension of SIMD where each thread has private state
SGPR	Scalar General-Purpose Register — holds values uniform across a wavefront
SM	Streaming Multiprocessor — NVIDIA's equivalent of AMD's Compute Unit
TDP	Thermal Design Power — maximum sustained power a GPU is rated for
TFLOPS	Tera Floating-Point Operations Per Second — measure of GPU compute throughput
TSV	Through-Silicon Via — vertical electrical connection through a silicon die (used in HBM stacking)
VGPR	Vector General-Purpose Register — holds per-thread values across a wavefront's 64 lanes
VRF	Vector Register File — the set of all VGPRs in a Compute Unit
wavefront	AMD term for the fundamental parallel execution unit (64 work-items executed in lockstep)

References & Further Reading

The following resources are recommended for deeper exploration of each topic covered in this document:

AMD Official Documentation

- AMD ROCm Documentation: <https://rocm.docs.amd.com>
- AMD Instinct MI300X Product Page and Technical Overview (AMD, 2023)
- AMD CDNA3 Architecture Whitepaper (AMD, 2023)
- HIP Programming Guide: <https://rocm.docs.amd.com/projects/HIP>
- AMD Omnipert Documentation: <https://amdresearch.github.io/omnipert>

Academic & Technical References

- Williams, S., Waterman, A., Patterson, D. (2009). Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures. Communications of the ACM, 52(4), 65–76.
- Nvidia CUDA C++ Programming Guide (applicable for contrast and comparison with HIP)
- Goodfellow, I., Bengio, Y., Courville, A. (2016). Deep Learning. MIT Press.
- Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS 2017. (Foundation of Transformer architecture)

Online Learning Resources

- AMD GPU Developer Conference (AMDGPU Conf) session recordings — available on AMD YouTube channel
- Triton Lang Documentation: <https://triton-lang.org> — for writing custom GPU kernels
- GPU MODE Discord & Lectures: community resources on GPU kernel optimization (<https://gpumode.com>)
- Simon Boehm's 'How to Optimize a CUDA Matmul Kernel' — excellent practical GEMM optimization guide applicable to HIP

End of Document

AMD GPU Fundamentals & AI Workloads — Personal Study Reference | May 2026